Domain Specific Language Design (2IMP20): Syntax

Loek Cleophas

Ivan Kurtev

Department of Mathematics and Computer Science — Software Engineering and Technology cluster

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

Defining DSLs — Grammarware vs. Modelware

Grammarware:

- Language definitions based on *lexical and context-free syntax definitions*

Modelware:

- Language definitions based on *meta-models*

Defining DSLs in the grammar world

Goal: *defining languages* and *manipulating programs expressed in them*

Rascal language and environments

- VS Code based IDE for defining languages i.e. Language Workbench
  - An interactive development environment *for defining formal languages and generating tools from them*
  - See www.rascal-mpl.org

Defining DSLs in the grammar world

Reading material:

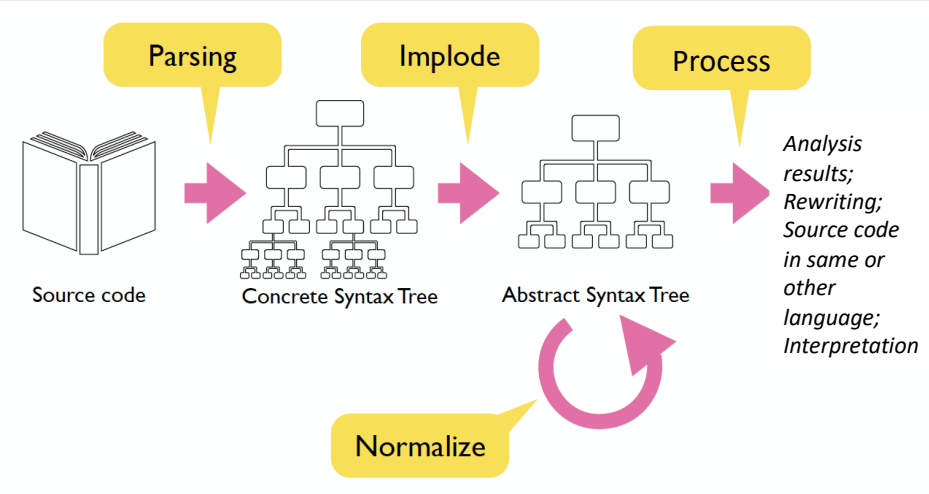
- Chapter 1 of "Introduction to Compiler Design" by Torben Mogensen (see https://link.springer.com/book/10.1007/978-0-85729-829-4, via TU/e library)
- Online material on Rascal: https://www.rascal-mpl.org/docs/Rascal/Declarations/SyntaxDefinition/ (*and see first lecture's slides*)

This + part of next lecture by Loek:

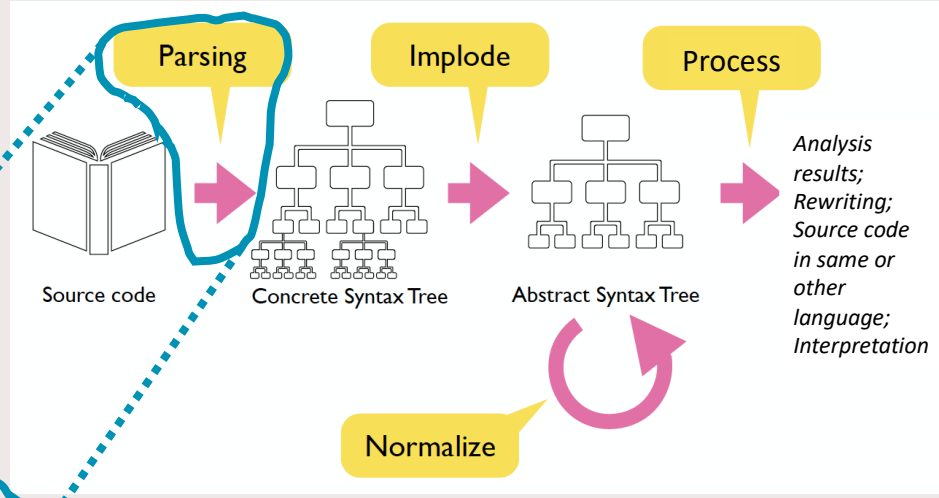
- Syntax definition and syntactical analysis
  - Lexical + context-free syntax
  - Concrete + abstract syntax

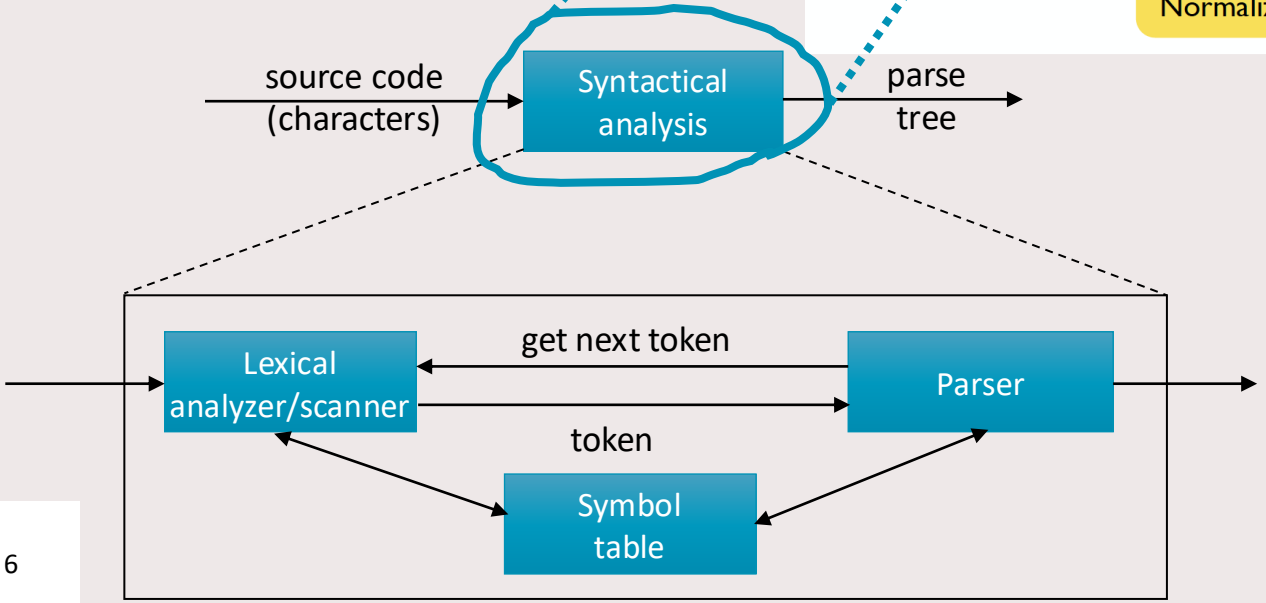
Later: semantics

Syntactical analysis a.k.a. parsing



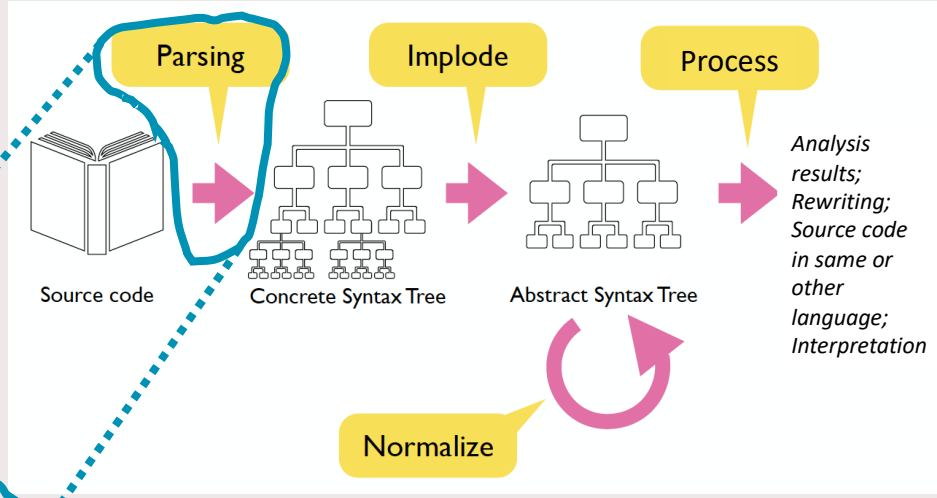
Syntactical analysis a.k.a. parsing

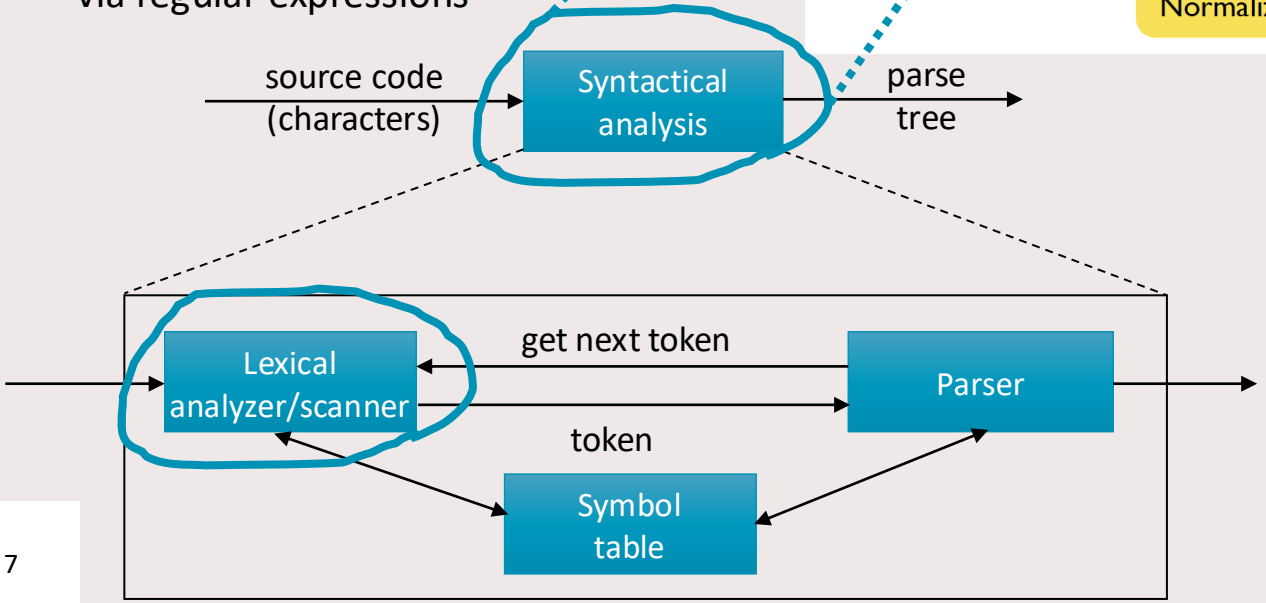




Lexical analysis a.k.a. scanning

- Tasks and organization of *lexical analyzer* a.k.a. *scanner*
- Specification of lexical *tokens* via regular expressions





Lexical syntax

Tasks of the lexical analyzer:

- reading the input and producing of tokens
  - each token represents one *logical piece of the source* – keyword, name of a variable, value, etc. (corresponds to a *syntactic category*)
  - each token may have *optional attributes*
    - including e.g. a *lexeme* (e.g. “myVar”, “+”)

- elimination of layout and comments?
- keeping track of position information? → line and column attributes

Lexical syntax

What are relevant tokens?

```
for (int k = 0; k < myArray[5]; ++k) {
 cout << k << endl;
}
```

```
for             {
int             }
cout            ;
<<              <
=               [
(               ]
)               ++

Identifier
IntegerConstant
```


TU/e

Lexical syntax

Scanning is not trivial, especially in “old” languages

- FORTRAN: whitespace is irrelevant

```
DO I = 1,25
DOI = 1,25
DOI = 1.25
```

  - First and second line represent a loop running from i = 1 to 25
  - Third line is an assignment of the value 1.25 to the variable DOI

→ Difficult to tell how to partition the input